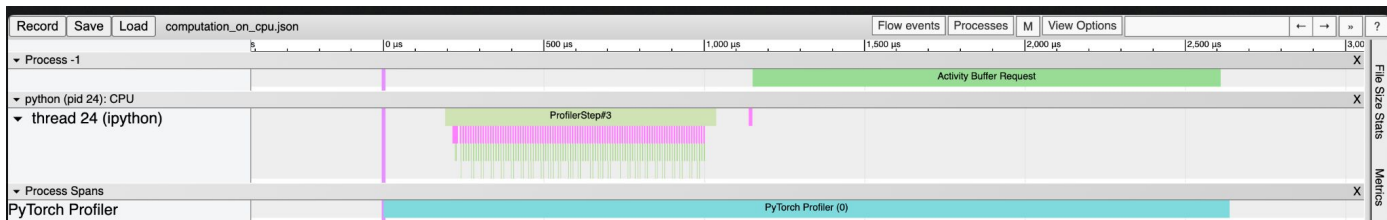


When the operation is super small (like doing a matrix multiplication of two square matrices of size 10), doing it in the CPU itself is better than transferring it onto gpu and doing the computation there.

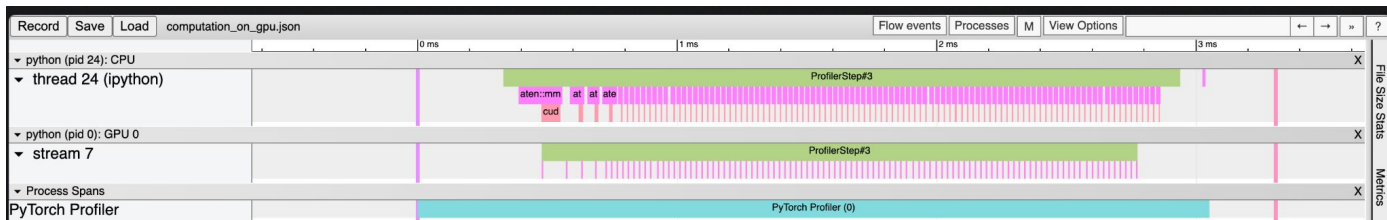
Computation on CPU

Each operation by CPU involving the matrix multiplication (meaning the whole computation) takes around .005ms. Total duration of all computations getting done is around .846ms



Computation on GPU

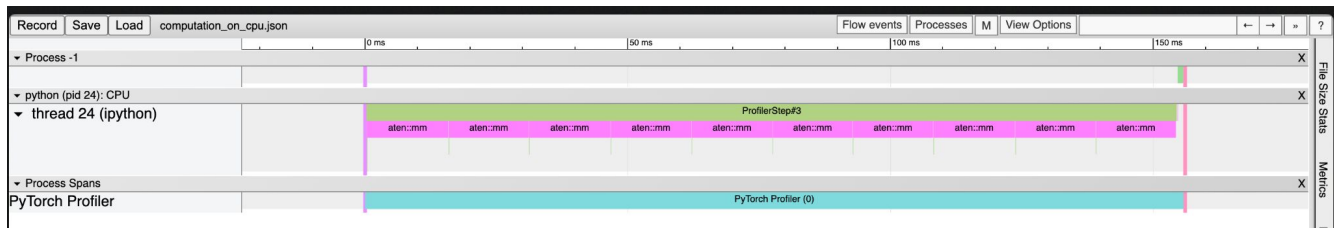
Each operation by CPU involving the matrix multiplication (which is basically sending to GPU) takes around .017ms. Total duration of all computations getting done is around 2.5ms



When multiplying 2 square matrices of size 1000, the time taken for computation on CPU is much more than time taken for computation on GPU

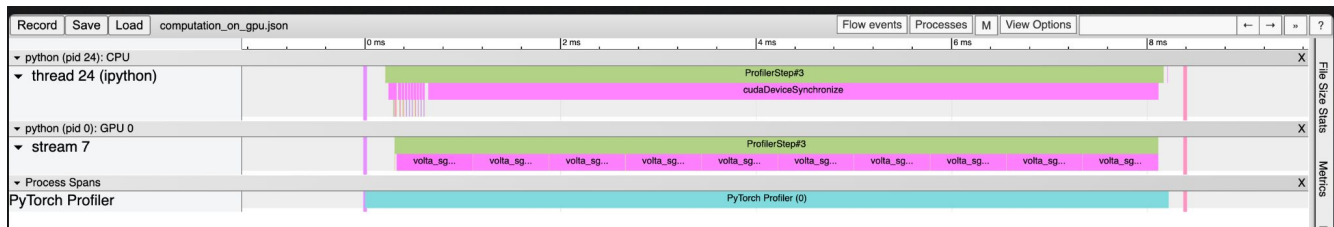
Computation on CPU

Each operation by CPU involving the matrix multiplication (meaning the whole computation) takes around 15ms. Total duration of all computations getting done is around 150ms



Computation on GPU

Each operation by CPU involving the matrix multiplication (which is basically sending to GPU) takes around .025ms. Total duration of all computations getting done is around 8ms



Side Note: Initially, the GPU trace wasn't visible for some reason (even after adding `torch.cuda.synchronize()` just before `prof.step()`). Then, I did 100 iterations instead of 10, and it reappeared. Then, post restarting the modal instance, and ipython kernel, it reappeared even with 10 iterations, so its a bit strange when it appears and when not.

In Conclusion: when we do heavy computations on CPU, it itself gets occupied and does the computations which takes a lot of time if they are heavy. However, if we divert them to GPU, the CPU takes around 0.025ms duration (atleast for the matrix multiplications I tested) in sending the task to GPU, irrespective of how big the task is, as it just has to send and not compute itself, and the GPU at some point in the future, executes them real fast.